AI-NATIVE GROCERY MERCHANT

Document 12 — Decision Log

Locked architectural, product and engineering decisions • Development reference

Document status: LOCKED DEVELOPMENT REFERENCE

This document records decisions that materially affect implementation. It exists to prevent previously resolved questions from being reopened during development without an explicit scope review.

1. Decision Governance

A decision is considered locked when it is consistent with the product scope, architecture and acceptance criteria and has been carried into the relevant development specifications.

Change rule

• Do not reverse a locked decision casually.

• A change must identify the affected documents, code modules, tests and demo behavior.

• Security, payment and authority-boundary changes require explicit review.

• A new feature is not justified merely because it is technically easy to add.

2. Core Product Decisions

ID	Decision	Rationale	Status
D-001	Product is AI-Native Grocery Merchant	Provides a focused, demonstrable AI-commerce use case.	LOCKED
D-002	AI reasons; backend remains authoritative	Prevents hallucinated financial state.	LOCKED
D-003	User chooses final basket	Best Value and Best Quality encode different preferences.	LOCKED
D-004	Policy engine authorizes money actions	Authorization must be deterministic and independent of LLM output.	LOCKED
D-005	Razorpay Test Mode only	Buildathon implementation must not use real money.	LOCKED
D-006	Modular monolith	Lower implementation complexity for solo MVP while preserving boundaries.	LOCKED
D-007	Controlled/seeded catalog	Makes quality, optimization and demo behavior reproducible.	LOCKED

3. Agent Authority Decisions

ID	Decision	Implementation consequence
D-008	Agent cannot modify mandate	Mandate mutation belongs outside agent tools.
D-009	Agent cannot set authoritative price	Prices come from commerce backend.
D-010	Agent cannot declare authoritative stock	Stock comes from commerce backend.
D-011	Agent cannot calculate final payable authoritatively	Backend quote service owns arithmetic.
D-012	Agent cannot call Razorpay directly	Payment adapter is backend-only.
D-013	Agent cannot mark payment successful	Only verification/webhook flow changes payment state.
D-014	Agent receives structured policy denial	Supports safe re-optimization without policy override.
D-015	Product/review content is untrusted	Prompt/tool injection must not influence authority.

4. Basket Strategy Decisions

ID	Decision	Rule
D-016	Generate Best Value	Lowest practical cost while satisfying requirements and acceptable quality.
D-017	Generate Best Quality	Highest practical quality/reliability within authorized budget.
D-018	Budget is a ceiling	Unused budget is not a reason to add products.
D-019	User may override AI recommendation	Recommendation is preference guidance, not authorization.
D-020	Exact requirements are mandatory	Optimization cannot drop required quantities to fit budget silently.
D-021	Quality can beat lowest price	A cheap product below required quality is not feasible.

5. Quantity and Pack Decisions

Pack optimization is treated as a real optimization problem rather than a simple lowest-unit-price lookup.

ID	Decision	Example
D-022	Evaluate pack combinations	6 eggs can be one 6-pack or three 2-packs.
D-023	Avoid excessive surplus	Do not buy substantially more than required without justification.
D-024	Compare effective cost and quality	A combination may beat a single pack.
D-025	Stock is a hard constraint	Unavailable packs cannot form an authorized basket.

6. Smart Incentive Decisions

ID	Decision	Rule
D-026	Voucher is not automatically applied	Evaluate actual user value.
D-027	Voucher supports USE_NOW	Use when current benefit is meaningful and valid.
D-028	Voucher supports SAVE_FOR_LATER	Preserve when current benefit is weak and future utility is meaningful.
D-029	Voucher supports DO_NOT_USE	Use none when invalid, ineligible or not beneficial.
D-030	Never add unnecessary items for incentives	Basket requirements remain primary.
D-031	Loyalty follows same value principle	Current redemption must be weighed against future utility.

7. Quality and Evidence Decisions

• Quality claims must be supported by controlled evidence in the MVP.

• Unverified allegations must not be presented as facts.

• Conflicting evidence should reduce confidence rather than trigger invented certainty.

• Quality should influence optimization when the requirement makes quality material.

• The demo may use seeded evidence fixtures rather than an unrestricted web crawler.

8. Policy and Mandate Decisions

ID	Decision	Rule
D-032	Mandate defines authority	Agent cannot expand it.
D-033	Final payable enforces max spend	Gross amount alone is not the final authorization amount when valid discounts apply.
D-034	Evaluation order is fixed	Mandate → category → stock → per-item → incentive → amount → max spend → ALLOW/DEN
D-035	Policy fails closed	Uncertain authorization does not become ALLOW.
D-036	Policy runs before Razorpay	No provider order before ALLOW.
D-037	Policy revalidates before payment	Planning decisions can become stale.
D-038	Denial is structured	Reason code and context support recovery.

9. Payment Decisions

ID	Decision	Rule
D-039	Razorpay Test Mode is execution layer	Application remains authority.
D-040	Order amount comes from backend quote	Client amount is untrusted.
D-041	Payment success is server-confirmed	Client callback alone is insufficient.
D-042	Webhook signatures are validated	Invalid signatures cause no financial state mutation.
D-043	Webhook processing is idempotent	Duplicate events must not duplicate state.

ID	Decision	Rule
D-044	Payment amount must reconcile	Mismatch fails closed and requires reconciliation.

10. Audit Decisions

• Audit material AI decisions.

• Audit basket generation and recommendation.

• Audit user selection.

• Audit fresh quote.

• Audit policy ALLOW/DENY and reason.

• Audit Razorpay order/payment identifiers and state.

• Audit webhook processing.

• Preserve a correlation identifier across the money path.

• Historical authorization facts must not be reconstructed from mutable AI output.

11. Failure-Recovery Decisions

ID	Scenario	Required behavior
D-045	Basket exceeds mandate	DENY; do not create Razorpay order.
D-046	Agent receives policy denial	Re-optimize within unchanged mandate.
D-047	Price changes	Fresh quote; re-authorize.
D-048	Stock disappears	Reject/re-optimize.
D-049	Voucher expires	Recalculate incentive decision.
D-050	Checkout request repeats	Idempotent; no duplicate provider order.
D-051	Webhook repeats	Idempotent; no duplicate state transition.

12. Scope Decisions

The following are deliberately excluded from the locked MVP:

• Multi-merchant marketplace.

• Multi-agent architecture.

• Voice commerce.

• Negotiation engine.

• Production payments.

• Autonomous refunds/payouts/banking.

• Full ACP/AP2/UAP certification or implementation.

• Unrestricted shopping crawler.

• Large-scale recommendation infrastructure.

• Full loyalty platform.

• Generic coupon marketplace.

• Complex CRM/ERP/analytics integration.

• Large-scale concurrency infrastructure.

13. Decision Impact Map

Area	Locked decision that governs it
Frontend	User selection is explicit; frontend cannot authorize or set payment amount.
Agent	Reasoning is broad, financial authority is narrow.
Catalog	Server-owned price/stock/category data.
Optimization	Dual baskets, quality-aware packs, smart incentives.
Policy	Deterministic authorization with fixed checks.
Checkout	Fresh quote + policy ALLOW required.
Razorpay	Test Mode execution after authorization.
Audit	Full trace from goal to payment.
Testing	Document 10 verifies every critical boundary.

14. Reopening a Decision

A locked decision may be reconsidered only when one of the following is true:

• A requirement in the official buildathon track makes the decision invalid.

• A security or payment-integrity issue is discovered.

• A dependency makes the decision technically impossible.

• Testing reveals that the decision fails a locked acceptance criterion.

• The user explicitly requests a scope review.

Any approved change must update this decision log and all affected specifications before implementation proceeds.

15. Final Decision Summary

Central design: The product demonstrates agentic commerce through substantial AI research and optimization, while maintaining a hard separation between intelligence and financial authority.

Final money path: AI planning → user basket choice → fresh server quote → deterministic policy ALLOW → Razorpay Test Mode → server verification/webhook → audit.

Scope principle: Build the smallest complete system that proves this architecture reliably rather than expanding into adjacent commerce infrastructure.

16. Related Documents

Document	Dependency
01 — Development Master Spec	Defines locked product scope.
02 — Functional Requirements	Defines behavior affected by decisions.
03 — System Architecture	Defines component boundaries.
04 — AI Agent Specification	Defines agent authority and tools.
05 — Optimization Decision Specification	Defines optimization decisions.
06 — Policy & Mandate Specification	Defines authorization rules.
07 — Data Model Specification	Defines persistence of decisions.
08 — API Contract Specification	Defines implementation boundaries.
09 — Razorpay Integration Specification	Defines provider execution.
10 — Testing & Acceptance Specification	Defines proof that decisions hold.

Document	Dependency
11 — Development Roadmap	Defines implementation order.